ECE 310
Lab 8 Report: Sequential 8x8 Multiplier

Ohm Patel

November 16, 2025

Abstract

This report presents the design, implementation, and verification of an 8-bit by 8-bit sequential unsigned multiplier based on the shift-and-add algorithm. Unlike a combinational multiplier, the sequential design reuses a single adder across eight clock cycles, minimizing hardware resources while still producing a correct 16-bit product. The multiplier accepts new operands on a synchronous reset signal, then iteratively shifts and accumulates partial products until completion.

The design was implemented in Verilog, simulated using two non-trivial test vectors, and validated using waveform inspection in Vivado. Synthesis results confirmed the expected sequential architecture consisting of registers, a 16-bit adder, and simple control logic. Verification through simulation demonstrated correct functional behavior across all test cases. This lab reinforces concepts of sequential datapath design, register-transfer-level modeling, and hardware-efficient multiplication.

1 Introduction and Background

Binary multiplication can be implemented either combinationally, using a full parallel multiplier circuit, or sequentially, reusing hardware across multiple clock cycles. Sequential multipliers trade latency for reduced resource usage and simpler datapaths, making them ideal for constrained hardware environments.

This lab required the design of an 8x8 unsigned sequential multiplier using the shift-and-add algorithm. The algorithm operates by examining the least significant bit of the multiplier each cycle, conditionally adding the multiplicand to the running product, shifting the multiplicand left, shifting the multiplier right, and decrementing an internal counter until all eight iterations are complete. The multiplier loads new operands on a synchronous reset, and after eight cycles, the final 16-bit product is available for use.

The objective of the lab was to:

1. Implement the shift-and-add multiplier in Verilog using sequential logic.
2. Simulate the multiplier using a provided clock and multiple test vectors.
3. Capture waveforms demonstrating correct operation.
4. Compare the synthesized RTL structure to the expected hand-designed datapath.

2 Design and Block Diagram

The 8x8 multiplier is composed of three primary registers and a small control unit:

- A 16-bit `multiplicandReg` that shifts left each cycle.
- An 8-bit `multiplierReg` that shifts right each cycle.
- A 16-bit `product` register that accumulates partial sums.
- A 4-bit counter that tracks eight multiplication cycles.

On reset, the multiplicand is zero-extended to 16 bits, the multiplier is stored, the product is cleared, and the counter is set to eight. Each clock cycle, the design:

1. Checks the LSB of the multiplier.

2. Adds the multiplicand to the product if the LSB is 1.
3. Shifts the multiplicand left by one.
4. Shifts the multiplier right by one.
5. Decrements the cycle counter.

Figure 1 shows the full datapath used in the implementation.

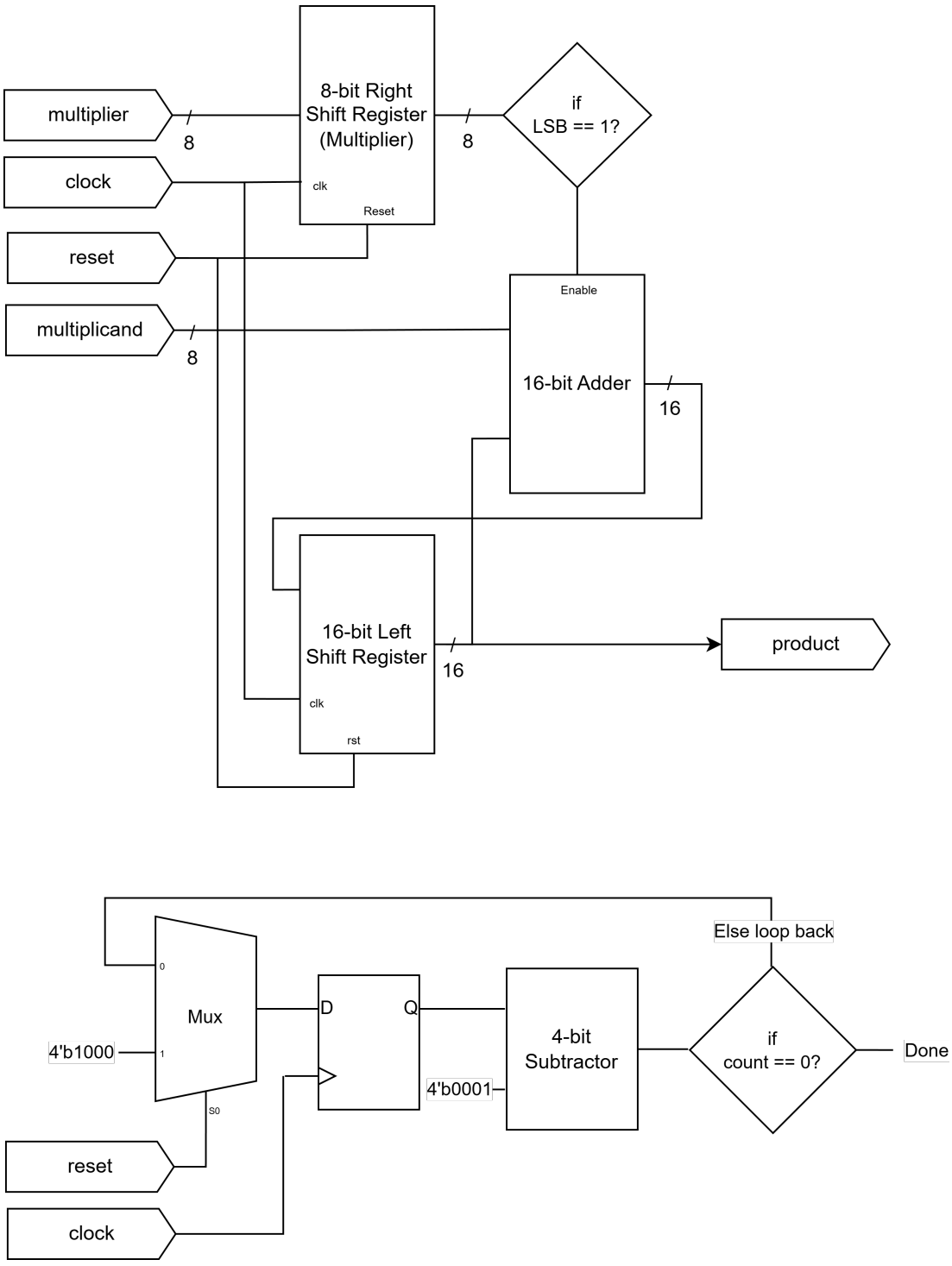

Figure 1: Datapath diagram of the sequential 8x8 multiplier.

3 Verilog Implementation

3.1 Multiplier Module

Listing 1: Verilog implementation of the 8x8 sequential multiplier

```
module mult_seq_8x8(
    input  [7:0] multiplier, multiplicand,
    output reg [15:0] product,
    input reset, clock
);

    reg [7:0] multiplierReg;
    reg [15:0] multiplicandReg;
    reg [3:0] count;

    always @(posedge clock) begin
        if (reset) begin
            product <= 16'b0;
            multiplierReg <= multiplier;
            multiplicandReg <= {8'b0, multiplicand};
            count <= 4'd8;
        end
        else if (count > 0) begin
            if (multiplierReg[0]) begin
                product <= product + multiplicandReg;
            end
            multiplicandReg <= multiplicandReg << 1;
            multiplierReg <= multiplierReg >> 1;
            count <= count - 1;
        end
    end

endmodule
```

3.2 Testbench

Listing 2: Testbench used to verify the sequential multiplier

```
module mult_seq_8x8_tb;

    mult_seq_8x8 dut(multiplier, multiplicand, product, reset, clock);
    initial clock = 0;
    always #5 clock = ~clock;

    initial begin
        reset = 0;  multiplier = 8'd0;  multiplicand = 8'd0; #20;

        // Test vector 1: 123 * 45 = 5535
        multiplier   = 8'd123;  multiplicand = 8'd45;  #10;
        reset = 1; #10; reset = 0; #100;
        $display("TV1: %d * %d = %d (expected 5535)", multiplier, multiplicand, product);
              #20;

        // Test vector 2: 200 * 200 = 40000
        multiplier   = 8'd200;  multiplicand = 8'd200;  #10;
        reset = 1; #10;  reset = 0;  #100;
        $display("TV2: %d * %d = %d (expected 40000)", multiplier, multiplicand, product)
            ;

        $finish;
    end

endmodule
```

4 Simulation Results

Two test vectors were used to validate the design:

1. $123 \times 45 = 5535$
2. $200 \times 200 = 40000$

The waveform clearly shows the eight-cycle multiplication process, including conditional addition based on the LSB of the multiplier, shifting of registers, and stabilization of the output product once the counter reaches zero.

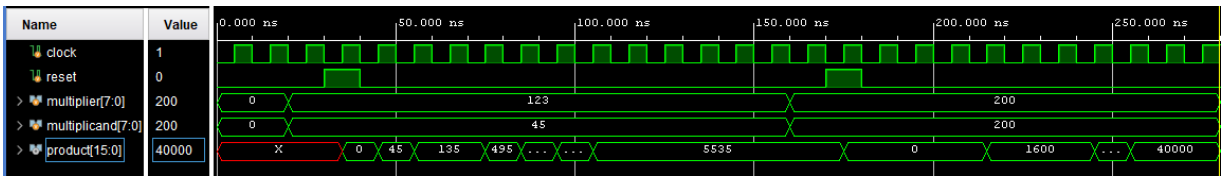

Figure 2: Simulation waveform showing both test vectors and correct product generation.

5 Synthesis Results and Discussion

Synthesis in Vivado produced a datapath matching the expected design:

- A single 16-bit adder.
- Three registers (product, multiplicand, multiplier).
- A small set of multiplexing and control logic.
- A 4-bit down-counter.

Compared to a combinational 8x8 multiplier, the sequential design uses significantly fewer LUTs and resources, at the cost of an 8-cycle latency. This tradeoff is consistent with the theoretical expectations for shift-and-add multipliers.

The synthesized RTL schematic confirmed:

1. Correct register-transfer structure.
2. Proper use of synchronous reset.
3. Correct operation of the shifter and adder network.

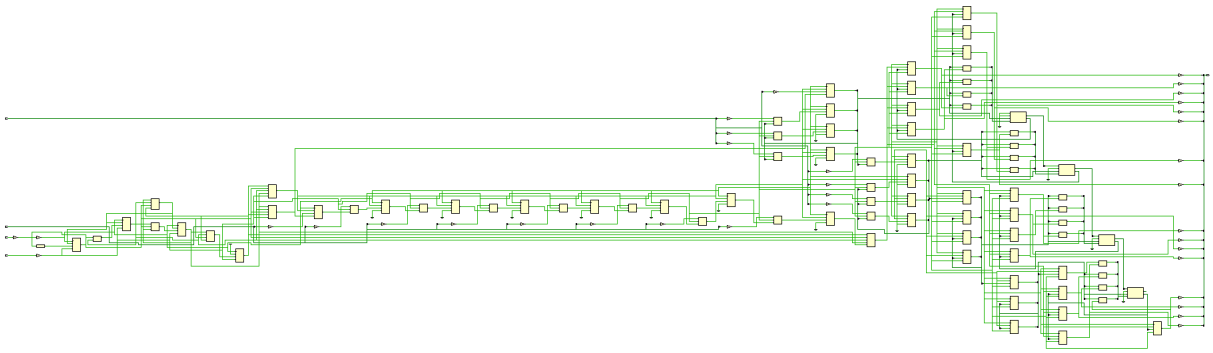

Figure 3: Synthesized RTL schematic of the sequential 8x8 multiplier.

6 Conclusion

An 8x8 sequential multiplier was successfully designed, implemented, simulated, and synthesized using the shift-and-add algorithm. The implementation met all functional requirements and produced correct results for all test vectors. Waveform inspection confirmed proper sequential behavior, including addition on LSB detection and correct register shifting. Synthesis results validated a hardware-efficient architecture that matched the expected hand-designed datapath. This lab reinforced key concepts in sequential digital system design and provided practical experience verifying RTL implementations in Vivado.

This report was typeset in LaTeX.